



Iqra University
Department of Computing & Technology

DevDialogue AI

Requirement-Aware AI Code Generation System

A Final Year Project Report

Submitted by

Syed Wafa Abbas Naqvi (37249)

Shahryar Mobusher (37189)

Shehryar Khan (37045)

Supervised by

Sumaira Aslam

In Partial Fulfillment of the Requirements for the Degree of
Bachelor of Science in Computer Science (BSCS)

Batch 2022–2026

FINAL APPROVAL

This is to certify that the Final Year Project report entitled “DevDialogue AI: A Requirement-Aware AI Code Generation System”, prepared and submitted by Syed Wafa Abbas Naqvi (37249), Shahryar Mobusher (37189), and Shehryar Khan (37045), has been examined and approved in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Computer Science (BSCS) at the Department of Computing and Technology, Iqra University, H-9 Campus, Islamabad, Pakistan.

The work presented in this report has been carried out in accordance with the academic standards and requirements prescribed by the University and is hereby recommended for evaluation.

Project Supervisor _____

Internal Examiner _____

External Examiner _____

Head of Department _____

Department of Computing and Technology
Iqra University, H-9 Campus, Islamabad, Pakistan

DEDICATION

We dedicate this work to Almighty Allah, whose blessings, mercy, and guidance gave us the strength, patience, and determination to complete this project.

To our beloved parents and families, whose endless love, prayers, and sacrifices have been the foundation of our academic journey, and whose encouragement has carried us through every challenge.

To our respected teachers and supervisor, whose guidance, knowledge, and support shaped our learning and made this work possible.

And to our friends and classmates, for their cooperation, motivation, and companionship throughout this journey.

With gratitude and respect,

Syed Wafa Abbas Naqvi
Shahryar Mobusher
Shehryar khan

ABSTRACT

Modern AI-assisted coding tools such as ChatGPT, Gemini, and DeepSeek generate code directly from a single, often incomplete prompt. Because these tools make no attempt to confirm that a request is complete before producing output, developers are pushed into a repetitive trial-and-error loop: they copy generated code into an editor, discover missing pieces or errors, revise the prompt, and repeat. This wastes time, increases cognitive load, and erodes trust in AI tooling. The underlying gap is the absence of a structured, dialogue-driven mechanism that collects complete requirements before any code is written.

DevDialogue AI is a web-based platform that closes this gap by placing a structured requirement-elicitation conversation in front of code generation. The system supports two complementary workflows. Mode 1 (Dialogue-Driven) conducts a fixed sequence of targeted questions covering the technology stack, core modules, authentication needs, and constraints, presents a consolidated requirements summary, and, on user confirmation, generates a complete set of project files. Mode 2 (Incremental) produces the project module-by-module, pausing after each file so the developer can approve it or request changes before the next module is generated.

The platform is implemented as a Django 6.0 web application following the Model-View-Template (MVT) pattern. It provides a full authentication subsystem with email One-Time-Password (OTP) verification, password reset, a custom password-strength policy, and subscription-tier profiles, together with persistent chat-style sessions, conversation history, and a code workspace. In the current implementation the requirement-elicitation dialogue and the two-mode workflow are fully functional, while the code-generation stage returns structured template output through a dedicated engine component whose interface is designed to be replaced by a large-language-model (LLM) service in future work.

The system was developed using an iterative, incremental methodology, and validated through twenty-seven test cases spanning unit, integration, and system levels, all of which passed. A deliberate design decision, isolating the generation stage behind a single interface, keeps the requirement-aware layer independent of the generation backend, so that a live model can later be substituted without disturbing the rest of the system. The project therefore demonstrates that a meaningful improvement in AI-assisted coding can come from a better interaction structure and not only from a more powerful model.

The system contributes to SDG 04 (Quality Education), SDG 08 (Decent Work and Economic Growth), SDG 09 (Industry, Innovation and Infrastructure), and SDG 17 (Partnerships for the Goals).

TABLE OF CONTENTS

ABSTRACT	ii
TABLE OF CONTENTS	iii
Chapter 1 INTRODUCTION	7
1.1 Project Overview	7
1.2 The Unmet Need and Its Audience	7
1.3 Aims and Objectives	8
1.4 Scope	8
1.5 Contributions	8
1.6 Report Organisation	9
1.7 Significance and Sustainable Development Goals	9
1.8 Development Methodology	10
1.9 System Requirements	10
1.10 User Roles and Responsibilities	11
Chapter 2 PROBLEM DEFINITION	12
2.1 Problem Statement	12
2.2 The Trial-and-Error Loop	12
2.3 Root-Cause Analysis	13
2.4 Impact	13
2.5 Proposed Resolution	13
Chapter 3 LITERATURE REVIEW	14
3.1 Background	14
3.2 Theoretical Background and Key Concepts	14
3.3 Related Approaches to Prompt Quality	16
3.4 Comparative Analysis of Existing Tools	17
3.5 Gap Analysis	18
3.6 Rationale for This Project	18
Chapter 4 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)	19
4.1 Introduction	19
4.2 Overall Description	20

4.3 Specific Requirements	24
4.4 Verification	27
4.5 Appendices	27
Chapter 5 SOFTWARE DESIGN SPECIFICATION (SDS)	28
5.1 Design Methodology and Software Process Model	28
5.2 System Overview	28
5.3 Design Models	31
5.4 Data Design	37
5.5 Security Design	39
5.6 User Interface Design	39
5.7 Design Summary	39
Chapter 6 IMPLEMENTATION AND TESTING	44
6.1 Implementation Discussion	44
6.2 Tools and Technologies	44
6.3 Module Structure	45
6.4 Testing Strategy	46
6.5 Unit Test Cases	47
6.6 Integration Test Cases	48
6.7 System / Acceptance Test Cases	50
6.8 Test Summary	51
Chapter 7 CONCLUSION AND FUTURE WORK	52
7.1 Conclusion	52
7.2 Limitations	53
7.3 Future Work	53
7.4 Closing Remarks	53
REFERENCES	54

Chapter 1

Introduction

Artificial intelligence has reshaped how software is written. Tools such as GitHub Copilot, ChatGPT, and Gemini let a developer describe a feature in natural language and receive working code in return. Despite this progress, a persistent weakness remains: these tools respond to whatever prompt they are given at a single moment, with no structured step that verifies whether the request actually contains enough information to produce correct, complete software. The developer, not the tool, is left to notice what was missing.

The result is a familiar and frustrating cycle. A developer types a short description, receives code that is partially correct, identifies the gaps, rewrites the prompt, and repeats, sometimes many times, before arriving at something usable. Each iteration consumes time and mental effort, and the productivity that AI tooling promises is steadily eroded by the overhead of correcting under-specified output.

1.1 Project Overview

DevDialogue AI is a web-based platform that addresses this problem by inverting the usual order of operations. Instead of generating code from an unstructured prompt, the system first conducts a short, structured interview to elicit the requirements that matter, and only then produces code. The interview is delivered through a familiar chat interface: the developer describes the project, and the system responds with targeted questions about the technology stack, the core modules or features, the authentication and user-role needs, and any external constraints or edge cases. Once these are gathered, the system presents a consolidated requirements summary and, on confirmation, generates the project files into a side-by-side code workspace.

The platform provides two generation workflows. Mode 1 (Dialogue-Driven Generation) completes the full interview first and then delivers the entire set of files at once; it suits smaller, well-defined tasks. Mode 2 (Incremental Generation) builds the project one module at a time, pausing after each file so the developer can approve it or ask for changes before proceeding; it suits larger, multi-part projects where step-by-step control is valuable.

1.2 The Unmet Need and Its Audience

The need this project addresses is a structured requirement-gathering step for AI code generation, a mechanism that ensures the important questions are asked before code is produced, rather than discovered afterwards. This matters most to two groups. The first is student and early-career developers, who benefit from being guided to think about stack, modules, authentication, and

constraints in a disciplined way, and for whom the platform doubles as a lesson in requirements thinking. The second is professional developers prototyping new systems, who lose time to the trial-and-error loop and want the specification pinned down before generation begins.

1.3 Aims and Objectives

The central aim of DevDialogue AI is to demonstrate that a structured, dialogue-driven front end improves the quality of AI-assisted code generation compared with single-prompt interaction. The supporting objectives are:

To design and implement a conversational requirement-elicitation engine that asks context-relevant questions and consolidates the answers into a clear specification summary.

To support two distinct generation workflows, full and incremental, within a single, coherent session model.

To provide a secure, production-shaped web application, including account registration with email OTP verification, password reset, a custom password policy, and persistent session history.

To present generated output in an accessible code workspace with per-file viewing, copying, and downloading.

To structure the generation stage behind a clean interface so that the current template-based engine can later be replaced with a large-language-model service.

1.4 Scope

The project is scoped to software requirement-elicitation and code-artefact generation delivered entirely through a web browser. It targets Python and general web-development projects as its primary use cases. It manages user accounts, persistent sessions, and conversation history, and it presents generated files for review and download. It excludes native mobile applications, hardware or embedded programming, offline or IDE-plugin usage, and real-time multi-user collaborative editing. The persistence layer uses SQLite in development, accessed through the Django ORM so that a production database can be substituted without application-level changes.

1.5 Contributions

The specific contributions of this project are fourfold. First, it delivers a working, deterministic requirement-elicitation dialogue that makes the specification explicit and reviewable before generation, rather than leaving it implicit in a prompt. Second, it provides two first-class generation workflows, full and incremental, unified under a single session model, so that the same platform serves both quick tasks and larger, staged builds. Third, it packages these ideas inside a complete and secure account and session subsystem that is shaped like a real product rather than a demonstration. Fourth, it adopts an architecture that isolates the generation stage behind a clean interface, so the value of the requirement-aware layer can be demonstrated

independently of the raw power of any particular generation backend, and a live model can later be substituted with minimal change.

Taken together, these contributions show that a meaningful improvement in AI-assisted coding can come not only from stronger models but also from a better interaction structure around them, an angle that is comparatively under-explored in current tools.

1.6 Report Organisation

The remainder of this report is organised into six further chapters. Chapter 2 defines the problem precisely and analyses its root cause and impact. Chapter 3 reviews existing AI code-generation tools and related research and identifies the specific gap that this project targets. Chapter 4 presents the Software Requirements Specification, following the IEEE 830 recommended practice, and covers the overall description together with the detailed functional and non-functional requirements. Chapter 5 presents the Software Design Specification, including the system architecture and the full set of design models, the data design, the security design, and the user-interface design. Chapter 6 describes the implementation and testing, including the detailed test cases and their results. Chapter 7 concludes the report and sets out the future work. The report closes with a consolidated list of references.

1.7 Significance and Sustainable Development Goals

The significance of this project extends beyond its immediate technical contribution. By lowering the effort required to translate an idea into a working starting point, a requirement-aware code-generation tool supports both learning and productivity. For students it makes the discipline of requirements thinking concrete and habitual; for practitioners it reduces the wasted effort of the trial-and-error loop and shortens the path from concept to a reviewable draft.

The project aligns with several of the United Nations Sustainable Development Goals. It contributes to SDG 04 (Quality Education) by turning an abstract software-engineering principle, gathering requirements before building, into an interactive tool that learners can practise with directly. It supports SDG 08 (Decent Work and Economic Growth) by reducing repetitive, low-value effort and letting developers focus on higher-value design and problem-solving. It advances SDG 09 (Industry, Innovation and Infrastructure) by exploring a novel interaction pattern for AI-assisted development. And it reflects SDG 17 (Partnerships for the Goals) through its use and integration of open technologies and its designed openness to third-party model services.

1.8 Development Methodology

The system was developed using an iterative, incremental methodology aligned with Agile practice. Rather than attempting to specify and build the whole system in a single pass, the work proceeded in short cycles, each of which delivered a working slice that could be tested before the next was added. The first cycle established the secure account subsystem, so that every subsequent feature could be developed behind real authentication. Later cycles added session management and the dialogue engine, then the two generation modes and the code workspace, and finally profile management and history features.

This ordering suited the project for two reasons. First, the conversational behaviour of the dialogue engine benefited from repeated refinement against actual use rather than a single up-front design; each cycle exposed rough edges in the question flow that were smoothed in the next. Second, building the security foundation first meant that access-control behaviour was exercised continuously throughout development rather than being bolted on at the end. Each increment concluded with testing at the unit and integration levels, so that regressions were caught early and the system remained in a working state throughout.

1.9 System Requirements

The development and operation of the system depend on modest, widely available hardware together with a standard open-source software stack. The requirements below describe the resources needed to build, run, and use the platform.

1.9.1 Hardware Requirements

- A development machine with an Intel Core i5 (or equivalent) processor.
- At least 8 GB of RAM.
- At least 10 GB of available storage for the toolchain and project.
- A stable internet connection for email delivery and package installation.
- A modern web browser on the client device.

1.9.2 Software Requirements

- Python 3 and the Django 6.0 framework.
- The project dependencies: python-docx, python-pptx, pypdf, and python-dotenv.
- SQLite for development (with the option of PostgreSQL in production through the ORM).
- A configured SMTP email account for OTP and password-reset delivery.
- A code editor such as Visual Studio Code, and Git for version control.

1.10 User Roles and Responsibilities

The system is designed around clearly separated roles so that access and responsibility are well defined. Each role has a specific set of permissions appropriate to its function.

1.10.1 Developer (Primary User)

The developer is the principal user of the platform. After registering and verifying an account, the developer creates project sessions, chooses between the two generation modes, answers the requirement dialogue, reviews the consolidated summary, and generates code. The developer can view, copy, and download generated files, and can manage their own history by renaming, searching, and deleting sessions, as well as updating their profile and password. All of a developer's data is private to that account.

1.10.2 Administrator

The administrator oversees the operation of the platform through Django's built-in administration site. This role is responsible for backend data management and for the overall integrity and security of the system's records. The administrator does not participate in the requirement dialogue; the role exists to support and maintain the platform rather than to use its generation features.

1.10.3 Email Service (External Actor)

Although not a human user, the SMTP email service is an external actor in the system's interactions. It delivers the one-time passwords that verify accounts and secure password resets, and is therefore essential to the authentication flows even though it takes no independent action of its own.

Chapter 2

Problem Definition

This chapter frames the problem DevDialogue AI solves. It moves from the general observation that AI code tools are hard to steer to a specific, technically grounded statement of what the problem is, why it exists, and what its consequences are when left unaddressed.

2.1 Problem Statement

Current AI-powered code-generation platforms generate code from vague or incomplete prompts, producing output that is over-engineered, missing key requirements, or structurally incorrect. Because no structured checkpoint confirms that the request is complete before generation, developers are forced into a repetitive cycle of generating, inspecting, correcting the prompt, and regenerating. The core problem is the absence of a structured, dialogue-driven mechanism that collects complete project requirements before code is generated.

2.2 The Trial-and-Error Loop

It is worth making the loop explicit, because it is the concrete manifestation of the problem. A single interaction that ought to be one step instead becomes a cycle of five, each carrying its own cost. The developer writes a brief prompt, which is inevitably under-specified; copies the resulting code into the editor and runs it; inspects the output and diagnoses what is missing; returns to the tool to re-describe the missing requirement; and repeats the whole cycle until the output is finally acceptable. The loop is not caused by any single poor output. It is a structural consequence of asking a stateless tool to satisfy an under-specified request, and it recurs no matter how capable the underlying model is.

Stage	What the developer does	Cost incurred
Initial prompt	Writes a brief description of the program	Under-specification; decisions left implicit
First output	Copies the code into the editor and runs it	Context switch between tool and editor
Inspection	Reads the output and identifies gaps	Cognitive load of diagnosing the problem
Corrective prompt	Re-describes the missing requirement	Time lost; risk of new gaps
Repeat	Iterates until acceptable	Compounding delay and eroding trust

Table 2.1 Anatomy of the trial-and-error loop

2.3 Root-Cause Analysis

The root cause is the stateless, single-turn nature of prompt-based generation. Conventional tools treat each request as an isolated event: they have no built-in notion of whether the information supplied is sufficient, they do not ask clarifying questions, and they maintain no evolving model of the project requirements across turns. They simply attempt to satisfy whatever text they are given.

This design is adequate for small, self-contained snippets, where a single sentence genuinely does contain everything needed. It breaks down for any non-trivial project, one that involves several modules, relationships between data entities, authentication, external services, or deliberate error handling, because a single prompt almost never captures all of those dimensions at once. The missing information is not discovered until the generated code is inspected, at which point the cycle restarts.

2.4 Impact

The consequences are concrete and measurable. Development time is consumed by iterative correction rather than productive building. Cognitive load rises because the developer must remember what has already been communicated and track what the tool has misunderstood. Generated code frequently requires substantial manual rework before it is usable. And repeated experiences of incomplete or incorrect output steadily undermine trust in AI tooling, which in turn limits its adoption for serious work.

2.5 Proposed Resolution

DevDialogue AI resolves the problem by introducing an explicit requirement-elicitation phase ahead of generation. A dialogue engine interviews the developer across the dimensions that most often go unstated, technology stack, core modules, authentication and user roles, and constraints or edge cases, and consolidates the responses into a reviewable summary. Generation proceeds only after the developer confirms that summary. This turns code generation from a reactive, single-shot guess into a proactive, requirement-aware process, and it gives the developer a clear checkpoint at which the specification is made explicit before any code is committed.

The benefits of this approach follow directly from the structure of the interaction:

It reduces the number of correction cycles by capturing the important requirements before generation.

It makes the specification explicit and reviewable, so misunderstandings surface before code is written rather than after.

It lowers cognitive load, because the developer answers targeted questions rather than trying to anticipate everything at once.

It teaches good requirements habits, which is especially valuable for student and early-career developers.

It remains valid as models improve, because the improvement lies in the interaction structure rather than in any one backend.

Chapter 3

Literature Review

This chapter surveys the landscape of AI code-generation tools, identifies their shared limitation, and establishes the gap that DevDialogue AI is designed to fill.

3.1 Background

Large language models trained on code, beginning with systems such as OpenAI Codex and the research on program synthesis that followed, demonstrated that natural-language descriptions could be translated into working programs with surprising fluency. This capability was rapidly productised: GitHub Copilot embedded it in the editor, conversational assistants such as ChatGPT and Gemini made it available in a chat window, and open-weight models such as DeepSeek Coder made high-quality generation affordable through an API. Across all of these, the interaction model is fundamentally the same, the user writes a prompt and the model produces code, and the quality of the result depends heavily on how complete that prompt happens to be.

A parallel and older body of work in software engineering, requirements engineering, studies exactly the opposite discipline: how to elicit, clarify, and validate what a system must do before it is built. DevDialogue AI sits at the intersection of these two areas, borrowing the elicitation discipline of requirements engineering and applying it as a front end to modern code generation.

3.2 Theoretical Background and Key Concepts

Several concepts underpin the design of DevDialogue AI. Understanding them clarifies both the problem the system addresses and the means by which it does so.

3.2.1 Large Language Models

A large language model is a neural network trained on very large corpora of text, and in the case of code models, of source code, so that it can predict and generate plausible continuations of a given input. When applied to programming, such a model can translate a natural-language description into working code. Its output quality, however, is bounded by the completeness of the description it is given: a model cannot supply requirements the user never stated. This dependence on complete input is precisely what motivates a requirement-elicitation layer in front of the model.

3.2.2 Natural Language Processing

Natural language processing is the branch of artificial intelligence concerned with enabling computers to interpret and produce human language. In the context of this project, natural-language techniques allow the system to interpret a developer's free-text answers to the requirement questions and to present its own questions and summaries in fluent, readable form. The requirement dialogue is, in effect, a structured natural-language exchange whose purpose is to convert an informal description into an explicit specification.

3.2.3 Requirements Engineering

Requirements engineering is the systematic process of discovering, documenting, and validating what a software system must do before it is built. Its established techniques, structured interviews, clarifying questions, and explicit validation of assumptions, are exactly the practices that single-prompt code tools omit. DevDialogue AI operationalises a lightweight form of requirements engineering: its fixed question set is a miniature structured interview, and its confirmation step is a validation checkpoint.

3.2.4 The Django Framework

Django is a high-level Python web framework that follows the Model-View-Template pattern and provides, out of the box, an object-relational mapper, an authentication system, form handling, and protection against common web vulnerabilities. Building on Django allowed the project to focus its effort on the novel requirement-elicitation behaviour rather than on re-implementing the standard machinery of a secure web application.

3.3 Related Approaches to Prompt Quality

Several techniques have emerged to improve the quality of the prompts given to code models. Prompt engineering encourages developers to write richer, more structured prompts by hand, and has produced a body of informal best practice around examples, role instructions, and step-by-step reasoning. Retrieval-augmented generation attaches relevant context automatically from a codebase or documentation, so the model sees more of the surrounding project. Agentic frameworks let a model plan across several steps and call tools, decomposing a task rather than answering in a single shot.

Each of these improves the input to a model, but none of them changes the fundamental interaction: the burden of knowing what to specify still rests with the user, and no explicit, human-reviewed statement of requirements is produced before generation begins. DevDialogue AI differs in kind rather than degree. Instead of enriching a prompt behind the scenes, it externalises the requirements into a visible, confirmable artefact created through guided questioning, so the developer both sees and approves the specification before any code is written.

3.4 Comparative Analysis of Existing Tools

The table below compares representative AI coding tools against the specific concern of this project, whether requirements are gathered before generation.

Tool / Application	Purpose	Strengths	Weaknesses
GitHub Copilot	IDE-integrated code completion	Deep IDE integration; broad language support	No pre-generation requirement gathering
ChatGPT	General LLM for chat and code	Highly capable; conversational context	No structured elicitation; loop persists
Gemini	Multimodal LLM for code	Strong reasoning; Google integration	Same single-prompt limitation
DeepSeek Coder	Open-weight code model	High accuracy; low-cost API	Needs already-complete prompts
Cursor IDE	AI-powered code editor	Codebase-aware, multi-file edits	No upfront requirement interview

Table 3.1 Comparative analysis of existing AI code-generation tools

3.5 Gap Analysis

Before summarising the gap, it is worth examining each representative tool in a little more detail, since the shape of the gap emerges from the pattern that recurs across all of them.

3.5.1 GitHub Copilot

Copilot is deeply integrated into the editor and offers fast, inline completions across a wide range of languages, which makes it excellent for accelerating the writing of code that the developer already knows they want. Its weakness, from the perspective of this project, is that it acts on whatever context is present without any structured step that confirms the request is complete. It accelerates typing rather than clarifying intent, so the burden of knowing what to specify remains entirely with the developer.

3.5.2 ChatGPT and Gemini

General conversational assistants such as ChatGPT and Gemini are highly capable and can hold context across a conversation, which makes them flexible partners for a wide variety of tasks. However, they respond to each prompt as given and do not, of their own accord, run a disciplined requirement interview before producing code. A developer can simulate this by asking the model to interview them, but the discipline is optional and easily skipped, so in practice the trial-and-error loop persists.

3.5.3 DeepSeek Coder and Cursor

Open-weight code models such as DeepSeek Coder achieve strong benchmark results at low cost, and editor-integrated tools such as Cursor bring codebase-aware, multi-file editing to the developer. Both are powerful, but both perform best when they are given an already-complete specification; neither includes a first-class step that helps the developer arrive at that specification in the first place. Their strength lies downstream of the gap this project addresses.

The pattern is consistent: comparing these tools against the needs of a requirement-aware workflow reveals the same set of gaps, summarised below alongside how DevDialogue AI addresses each.

Comparing these tools against the needs of a requirement-aware workflow reveals a consistent set of gaps, summarised below alongside how DevDialogue AI addresses each.

Area	Current State	Identified Gap	DevDialogue AI Solution
Requirement elicitation	Single-shot prompts	No structured pre-generation interview	Dialogue engine collects stack, modules, auth, constraints
Workflow flexibility	Single-turn only	No incremental mode	Mode 2 builds step-by-step with approval
Requirement checkpoint	Gaps found after generation	No confirmation step	Summary confirmed before code is produced
Accessibility	Often paid	Students priced out	Open-source Django stack, free Basic tier
Extensibility	AI logic coupled to UI	Hard to change backend	Generation isolated behind a service interface

Table 3.2 Gap analysis and DevDialogue AI contribution

3.6 Rationale for This Project

The reviewed tools are individually powerful, yet they share one architectural blind spot: none places a structured requirement-elicitation step ahead of generation, and none offers an incremental, approval-driven build mode as a first-class workflow. DevDialogue AI was undertaken to explore precisely this gap. Rather than competing with the underlying models on raw generation quality, it contributes an interaction layer, a disciplined dialogue that makes requirements explicit and reviewable before code is produced, and packages it in an accessible, open-source web application. The generation backend is isolated behind a service interface so that, as models improve, the same elicitation layer can drive them directly.

Chapter 4

Software Requirements Specification (SRS)

This chapter specifies the software requirements for DevDialogue AI following the structure of the IEEE 830 recommended practice, reflecting the system as implemented, with notes where a capability is designed as an extension point for future work.

4.1 Introduction

4.1.1 Document Purpose

This SRS defines the requirements for DevDialogue AI so that its developers, testers, and supervisor share a single, unambiguous understanding of what the system must do. It serves as the reference against which the design (Chapter 5) and the implemented system are validated.

4.1.2 Product Scope

DevDialogue AI is a browser-based platform that elicits software requirements through a structured conversation and then produces project code artefacts. It covers account management, requirement-elicitation dialogue, two generation modes, session persistence, and a code workspace. Code generation is currently served by a template-based engine; integration of a live large-language-model backend is out of scope for the current version and identified as future work.

4.1.3 Intended Audience

This specification is written for several audiences. The development team uses it as the definitive statement of what must be built and as the basis for the design in Chapter 5. The testers use it to derive the test cases in Chapter 6 and to judge whether the delivered system behaves correctly. The project supervisor and examiners use it to assess whether the requirements are complete, consistent, and appropriately scoped. Because these audiences have different concerns, the specification is kept precise and unambiguous, and each requirement is written so that it can be verified objectively.

4.1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
DevDialogue AI	The web-based requirement-aware code-generation platform described in this document
Mode 1	Dialogue-Driven Generation: full interview first, then the whole codebase at once
Mode 2	Incremental Generation: one module at a time, with user approval between modules

Dialogue Engine	Server-side component that drives the question flow, builds the summary, and returns generated code
OTP	One-Time Password: a six-digit code emailed to verify account ownership; valid for two minutes
MVT	Model-View-Template, the architectural pattern used by Django
ORM	Object-Relational Mapper: maps Python model classes to database tables
CSRF	Cross-Site Request Forgery; mitigated by Django tokens on state-changing requests
LLM	Large Language Model; intended to drive generation in future work
SRS	Software Requirements Specification (this chapter)

Table 4.1 Definitions, acronyms, and abbreviations

4.1.5 References

The full reference list appears at the end of this report.

4.1.6 Document Overview

Section 4.2 gives the overall description of the product; Section 4.3 specifies the detailed interfaces and requirements; Section 4.4 describes verification; Section 4.5 lists appendices.

4.2 Overall Description

4.2.1 Product Perspective

DevDialogue AI is a new, self-contained web application built on Django. It interacts with one external service, an SMTP mail server used to deliver OTP and password-reset emails. Internally it is organised into a presentation layer (Django templates and vanilla JavaScript), an application layer (Django views, authentication, and the dialogue engine), and a data layer (the Django ORM over a SQLite database). Its context is shown in Figure 4.1.



Figure 4.1 System context diagram

4.2.2 Product Functions

The product provides secure account registration with email OTP verification; login, logout, and OTP-secured password reset; creation and management of project sessions in two modes; a structured requirement-elicitation dialogue with persistent history; template-based generation of

project files with per-file view, copy, and download; and profile and password management. The use cases are summarised in Figure 4.2.

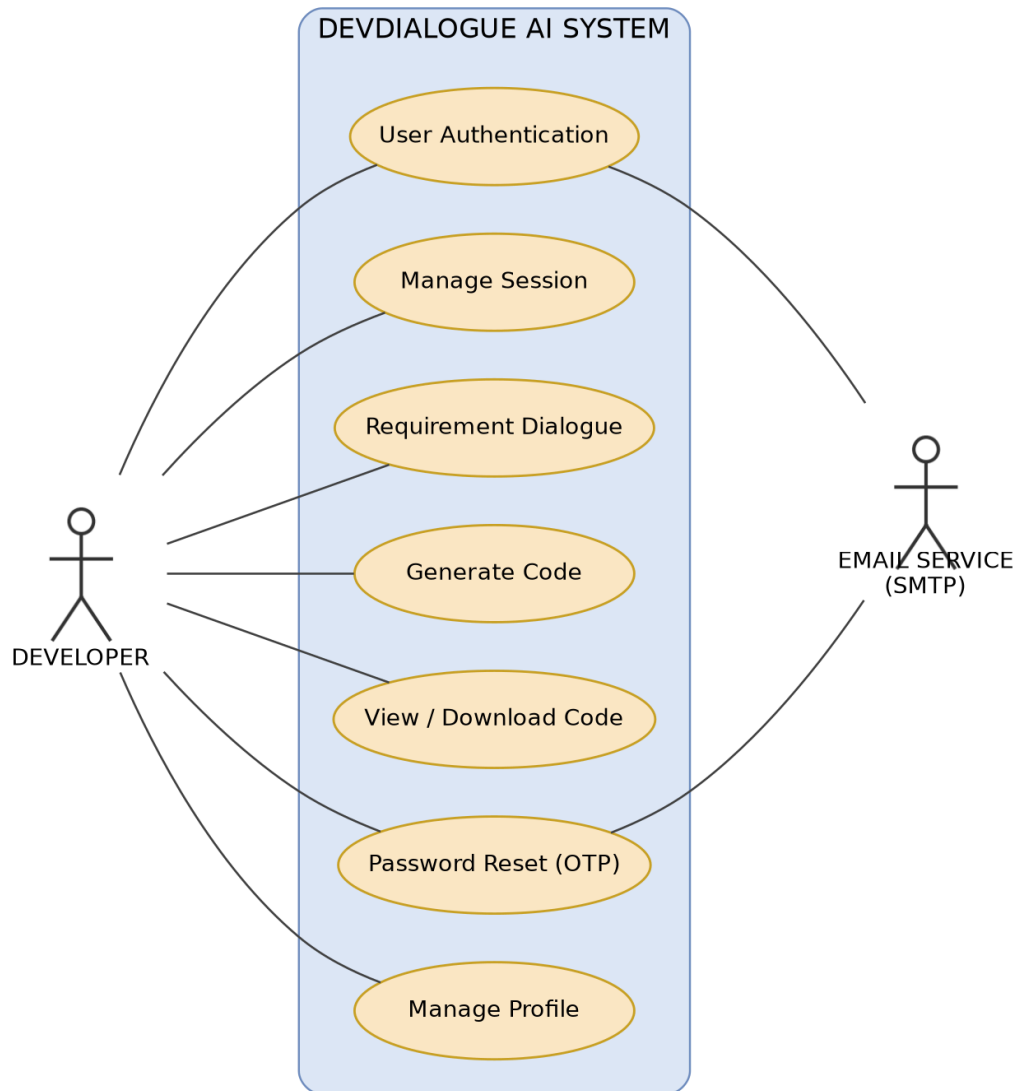


Figure 4.2 Use-case diagram

To illustrate the level of detail behind each use case, the central use case, the Mode 1 requirement dialogue and generation, is described fully in Table 4.2.

Field	Description
Use case	Conduct requirement dialogue and generate code (Mode 1)
Primary actor	Developer (authenticated, verified user)

Pre-conditions	User is logged in and email-verified; a new session has been created
Trigger	User submits an initial project description
Main flow	1. System asks the technology-stack question. 2. User answers. 3. System asks the modules question. 4. User answers. 5. System asks the authentication question. 6. User answers. 7. System asks the constraints question. 8. User answers. 9. System presents a consolidated requirements summary and asks for confirmation. 10. User confirms. 11. System generates the project files and displays them in the code panel; the session is marked completed.
Alternative flow	At step 10 the user requests changes instead of confirming; the system continues the dialogue rather than generating.
Post-conditions	Generated files are stored against the session and available to view, copy, and download.

Table 4.2 Detailed use-case description (Mode 1)

The remaining principal use cases are described in the same fully-dressed form in Tables 4.2a to 4.2d.

Field	Description
Use case	User Authentication (Login)
Primary actor	Developer, System
Description	The user supplies valid credentials; the system authenticates them and grants access to the dashboard.
Pre-conditions	The user has a registered, verified account.
Post-conditions	The user reaches their dashboard.
Main flow	1. User enters email and password. 2. System validates the credentials. 3. System confirms the account is verified. 4. System grants access to the dashboard.

Table 4.2a Use-case description — User Authentication

Field	Description
Use case	Register and Verify Account
Primary actor	Developer, System, Email Service
Description	A visitor registers, receives a one-time password by email, and verifies the account.
Pre-conditions	The email address is not already registered.
Post-conditions	The account is active and verified; the user is logged in.
Main flow	1. Visitor submits the sign-up form. 2. System creates an inactive account and profile. 3. System emails a six-digit OTP. 4. User enters the OTP within two minutes. 5. System activates and verifies the account.

Table 4.2b Use-case description — Register and Verify

Field	Description
Use case	Incremental Generation (Mode 2)
Primary actor	Developer, System
Description	After the interview, the system generates one module at a time, pausing for approval between modules.
Pre-conditions	The requirement summary has been confirmed in a Mode 2 session.
Post-conditions	All approved modules are stored and shown in the workspace.
Main flow	1. System generates the next module. 2. User reviews it. 3. User approves, or requests changes. 4. If changes are requested, the system regenerates that module with the feedback. 5. Steps repeat until every module is approved.

Table 4.2c Use-case description — Incremental Generation

Field	Description
Use case	Manage Sessions
Primary actor	Developer, System
Description	The user views past sessions, renames them, searches them, or deletes them.
Pre-conditions	The user is authenticated and owns at least one session.
Post-conditions	The session list reflects the requested change.
Main flow	1. User opens the session sidebar. 2. User searches or selects a session. 3. User renames or deletes it. 4. System applies the change and refreshes the list.

Table 4.2d Use-case description — Manage Sessions

4.2.3 Product Constraints

The system is delivered exclusively through a web browser; there is no native or offline client.

The backend is fixed to Django 6.0 on Python 3, using the MVT pattern.

The development database is SQLite; production databases are supported only through the ORM abstraction.

Email delivery depends on a correctly configured external SMTP service.

In the current version, generated code is template-based and deterministic rather than model-generated.

4.2.4 User Characteristics

Three user classes interact with the system. A Guest is an unauthenticated visitor who can browse marketing pages and access registration, login, and password reset. A Developer (the primary user) is an authenticated, email-verified account holder who creates sessions, answers

the dialogue, generates and downloads code, and manages their own history and profile. An Administrator uses Django's built-in admin site for backend data management.

4.2.5 Assumptions and Dependencies

A working SMTP account is configured through environment variables for OTP and reset emails.

Users have a modern browser with JavaScript enabled, since the chat workspace relies on client-side fetch requests.

The planned future capability depends on an external LLM (and optionally an N8N orchestration layer); the current system does not require them.

4.2.6 Apportioning of Requirements

Requirements are allocated across three subsystems: the authentication subsystem (FR-01 to FR-06, FR-16), the session and dialogue subsystem (FR-07 to FR-10, FR-14, FR-15, FR-17), and the generation subsystem (FR-11 to FR-13). Requirements deferred to future releases, chiefly live LLM-driven generation, are noted as such and are not part of the current acceptance criteria.

4.3 Specific Requirements

4.3.1 External Interfaces

4.3.1.1 User Interfaces

Responsive HTML templates styled with a custom CSS design system offering dark and light themes. Key screens are the landing pages, the authentication pages, the dashboard, and the chat workspace, which presents the conversation on the left and a collapsible, tabbed code panel on the right.

4.3.1.2 Software Interfaces

The application exposes internal JSON endpoints consumed by the browser: a send-message endpoint that drives the dialogue, a code-retrieval endpoint that returns a stored file's contents, and a rename endpoint. It depends on Django and the Django ORM, and on Django's mail subsystem for SMTP delivery.

4.3.1.3 Communications Interfaces

All client-server communication is over HTTP(S). Interactive requests use asynchronous fetch calls carrying JSON payloads and a CSRF token. Outbound email is sent over SMTP (default TLS on port 587).

4.3.2 Functional Requirements

ID	Functional Requirement
FR-01	The system shall allow a visitor to register with a username, email, and subscription plan, enforcing a custom password policy.
FR-02	The system shall deactivate a newly registered account until its email is verified via a six-digit OTP.
FR-03	The system shall email the OTP and accept it only within a two-minute validity window.
FR-04	The system shall allow the user to request a new OTP if the previous one has expired.
FR-05	The system shall authenticate registered users and reject invalid or unverified credentials.
FR-06	The system shall provide a password-reset flow secured by an emailed OTP.
FR-07	The system shall let an authenticated user create a new session and choose Mode 1 or Mode 2.
FR-08	The dialogue engine shall ask a structured sequence of requirement questions (stack, modules, authentication, constraints).
FR-09	The system shall persist every user and assistant message and display the full conversation history per session.
FR-10	The system shall present a consolidated requirements summary and request confirmation before generating code.
FR-11	In Mode 1, upon confirmation the system shall generate the complete set of project files at once.
FR-12	In Mode 2, the system shall generate one module at a time and require approval (or accept change requests) before continuing.
FR-13	The system shall store each generated file and expose it with per-file view, copy, and download.
FR-14	The system shall auto-generate a human-readable session title from the initial project description.
FR-15	The system shall let users rename and permanently delete their sessions.
FR-16	The system shall let users manage their profile (username, picture) and change their password.
FR-17	The system shall group session history by recency and support client-side search.

Table 4.4 Functional requirements

4.3.2.1 The Requirement-Elicitation Question Set

Requirement FR-08 is realised by a fixed, ordered set of four questions that together cover the dimensions most often left unstated in a single prompt. The dialogue engine advances through these in order, saving each answer, before assembling the summary required by FR-10. The set is given in Table 4.5.

#	Requirement dimension	What the question captures
1	Technology stack	Preferred languages, frameworks, and database (e.g. Python/Django, SQLite)
2	Core modules / features	The main features or modules the system should contain

3	Authentication / roles	Required user roles and authentication mechanisms (login, OTP, admin)
4	Constraints / edge cases	External APIs, database constraints, and error-handling or edge cases

Table 4.5 Requirement-elicitation question set

4.3.3 Quality of Service (Non-Functional) Requirements

ID	Category	Requirement
NFR-01	Performance	Dialogue responses shall return within a few seconds; the current engine responds effectively instantly as it performs no network calls.
NFR-02	Security	Authentication shall use Django; OTP codes expire after two minutes; passwords satisfy a custom policy; all state-changing requests are CSRF-protected.
NFR-03	Usability	A developer with no training shall complete the interview through a familiar chat interface; the UI supports dark and light themes.
NFR-04	Reliability	Session, message, and generated-code state shall be persisted so an interrupted session can be resumed without data loss.
NFR-05	Maintainability	The generation logic shall be isolated in one component so it can be replaced independently of the web and data layers.
NFR-06	Portability	The data layer shall be accessed through the ORM so the database can be changed without application-code changes.
NFR-07	Scalability	The stateless request/response design shall permit horizontal scaling of the web tier.

Table 4.6 Non-functional requirements

4.3.4 Compliance

This specification follows the IEEE 830 structure. The implementation follows Django conventions, including framework-managed authentication, CSRF protection on state-changing requests, and server-side password validation.

4.3.5 Design and Implementation Constraints

The system must be implemented in Python 3 on Django 6.0 following the MVT pattern; use the Django ORM for all database access; keep the code-generation logic behind a single engine interface; and rely only on the declared dependencies (Django, python-docx, python-pptx, pypdf, python-dotenv).

4.4 Verification

Requirements are verified by automated tests and manual demonstration. The repository includes unit tests for OTP generation, validity, and expiry; the custom password validator; the sign-up and reset forms; and the end-to-end dialogue flow through to generation. Detailed test cases appear in Chapter 6.

4.5 Appendices

Supporting design models are provided in Chapter 5; test cases in Chapter 6.

Chapter 5

Software Design Specification (SDS)

This chapter describes how DevDialogue AI is designed to satisfy the requirements of Chapter 4, covering methodology, architecture, design models, data design, security design, and user-interface design.

5.1 Design Methodology and Software Process Model

The system follows an object-oriented design approach, expressed through Django's model classes and the encapsulated dialogue-engine service. The development process followed an iterative, incremental model aligned with Agile practice: the project was built in short cycles that each delivered a working slice, first the authentication subsystem, then session management and the dialogue engine, then the two generation modes and the code workspace, and finally profile and history features. This let each layer be tested before the next was layered on top.

5.1.1 Design Assumptions

Users access the system through a modern web browser with JavaScript enabled.

A configured SMTP service is available for delivering OTP and password-reset emails.

Each developer works within their own account, and their sessions and data are private to that account.

The requirement dialogue follows a fixed four-question sequence sufficient for the target project types.

5.1.2 Design Constraints

The backend is fixed to Django 6.0 on Python 3, following the Model-View-Template pattern.

All database access is through the Django ORM, so the storage engine can be changed without application-code changes.

The generation logic must remain isolated behind a single interface to preserve its replaceability.

The current version generates deterministic, template-based output rather than model-generated code.

5.2 System Overview

DevDialogue AI is a three-layer Django web application. The presentation layer renders the workspace and account pages and communicates through asynchronous JSON requests. The application layer contains the views, the authentication and security components, and the dialogue engine. The data layer persists all state through the ORM over SQLite. A single

external dependency, the SMTP mail service, delivers verification and reset emails. The defining decision is the isolation of all generation logic inside one component, the dialogue engine, so the same interface that today returns template output can later call an LLM service without changing the surrounding layers.

5.2.1 Architectural Design

Figure 5.1 shows the layered architecture and the flow of control. The browser issues requests to the views; the views invoke the authentication components and the dialogue engine; both read and write domain state through the ORM; and the authentication components additionally call the SMTP service. The dashed element marks the extension point at which an LLM can be attached.

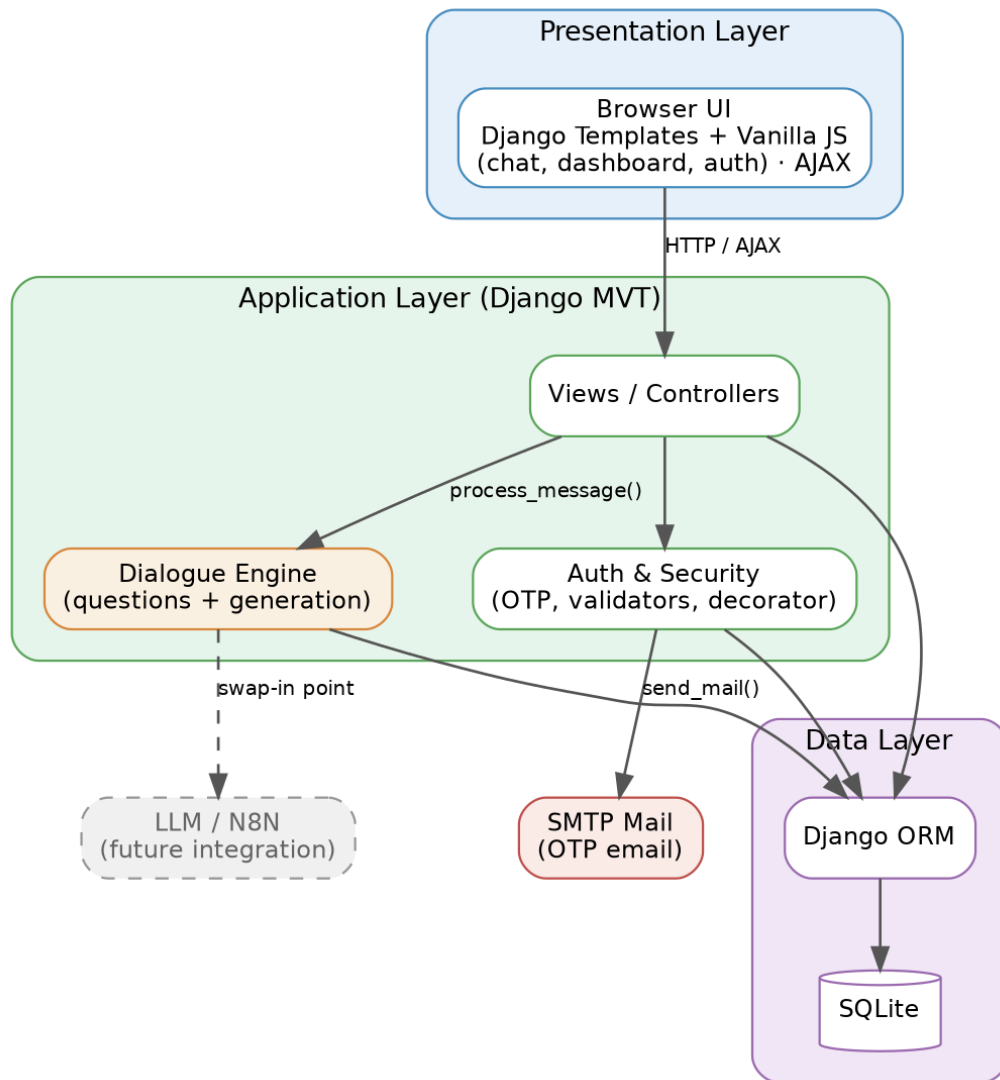


Figure 5.1 Layered architecture of DevDialogue AI

The presentation layer renders the interface and communicates with the server over asynchronous JSON requests.

The application layer holds the views, the authentication and security components, and the dialogue engine.

The data layer persists all state through the Django ORM over SQLite.

The SMTP service is the single external dependency, used for OTP and reset emails.

The dashed element marks the extension point where an LLM or N8N backend can be attached behind the engine.

Component	Technology	Responsibility
-----------	------------	----------------

Views / Controllers	Django function views	Handle HTTP and AJAX requests for auth, OTP, sessions, chat, and code retrieval
Auth & Security	Django auth, forms, validator, decorator	Registration, OTP, login/logout, reset, and access control
Dialogue Engine	core.dialogue_engine service	Drive the question flow, build the summary, and produce/store files
Session & History	Django ORM models	Persist sessions, messages, and generated code
Code Workspace	Django template + vanilla JS	Tabbed, collapsible panel with view/copy/download
Mail Service	Django mail over SMTP	Deliver OTP and password-reset emails
Data Store	SQLite via ORM	Durable storage for all entities

Table 5.1 Component-to-technology mapping

5.2.2 Request-Handling Design

A representative interactive request illustrates how the layers cooperate. When the developer sends a chat message, the browser issues an asynchronous POST carrying the message text and a CSRF token. The receiving view first enforces access control, the user must be authenticated and verified, then rejects the request early if the session is already completed. It parses the JSON body, delegates to the dialogue engine, and serialises the engine's response, together with the session status and the list of generated files, back to the browser as JSON. The browser then updates the conversation and, when files are present, refreshes the code tabs and loads the first file through a second endpoint. Because the view holds no state between requests, any number of requests can be served independently, which is what makes the web tier straightforward to scale.

5.3 Design Models

5.3.1 Class Diagram

Figure 5.2 shows the domain classes. Each user has exactly one Profile (created by a post-save signal) and owns any number of Sessions. Each Session owns an ordered collection of Messages and a collection of GeneratedCode records. The DialogueEngine is a stateless service that operates on these entities.

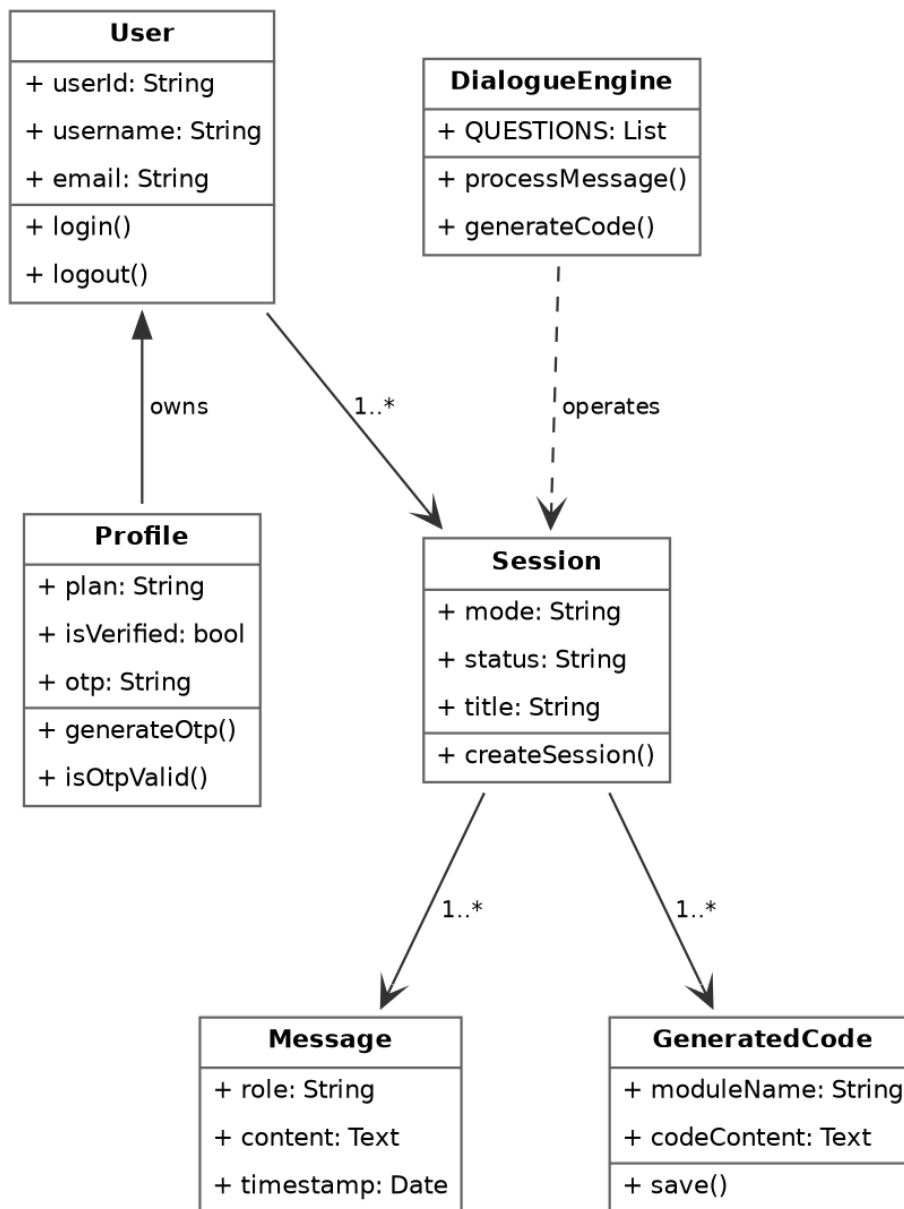


Figure 5.2 Class diagram

The User class holds the core account information and the login and logout behaviour. Each User has exactly one Profile, which carries the subscription plan, verification status, and OTP logic. A User owns any number of Sessions; each Session aggregates its Messages and GeneratedCode records.

The DialogueEngine is a stateless service that operates on these entities to drive the interview and generation.

5.3.2 The Dialogue Engine Design

The dialogue engine encapsulates the core behaviour. It holds a fixed, ordered question set (stack, modules, authentication, constraints). On each message it saves the message, counts how many questions have been asked, and either poses the next question or, once all are answered, assembles a summary and asks for confirmation. On confirmation it generates the files: Mode 1 produces the full set at once; Mode 2 emits one module at a time and waits for approval, regenerating with feedback if changes are requested. The file contents come from a deterministic template method, the single point at which a live model would later be substituted.

5.3.3 Data-Flow Diagrams

Figure 5.3 shows the context-level diagram; Figure 5.4 decomposes the system into its four principal processes and four data stores.

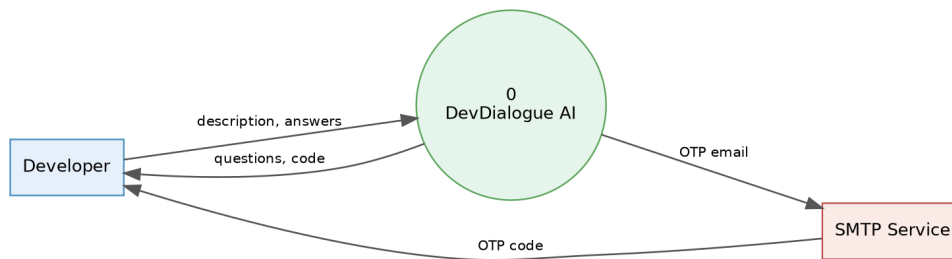


Figure 5.3 Data-flow diagram — Level 0 (context)

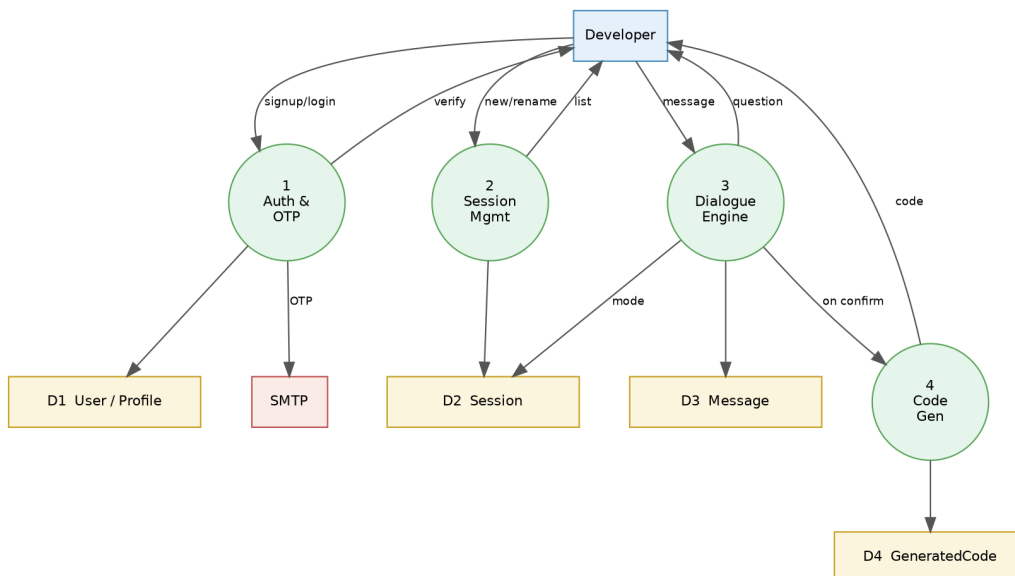


Figure 5.4 Data-flow diagram — Level 1

Process 1 handles authentication and OTP, reading and writing the User/Profile store.

Process 2 manages sessions, maintaining the Session store.

Process 3 is the dialogue engine, which records messages and, on confirmation, invokes generation.

Process 4 generates code and writes it to the GeneratedCode store.

5.3.4 Sequence Diagram

Figure 5.5 traces a Mode 1 interaction: the browser posts a message; the view delegates to the engine, which saves it, determines progress, and returns either the next question or the generated files; the browser then retrieves each file's contents to display.

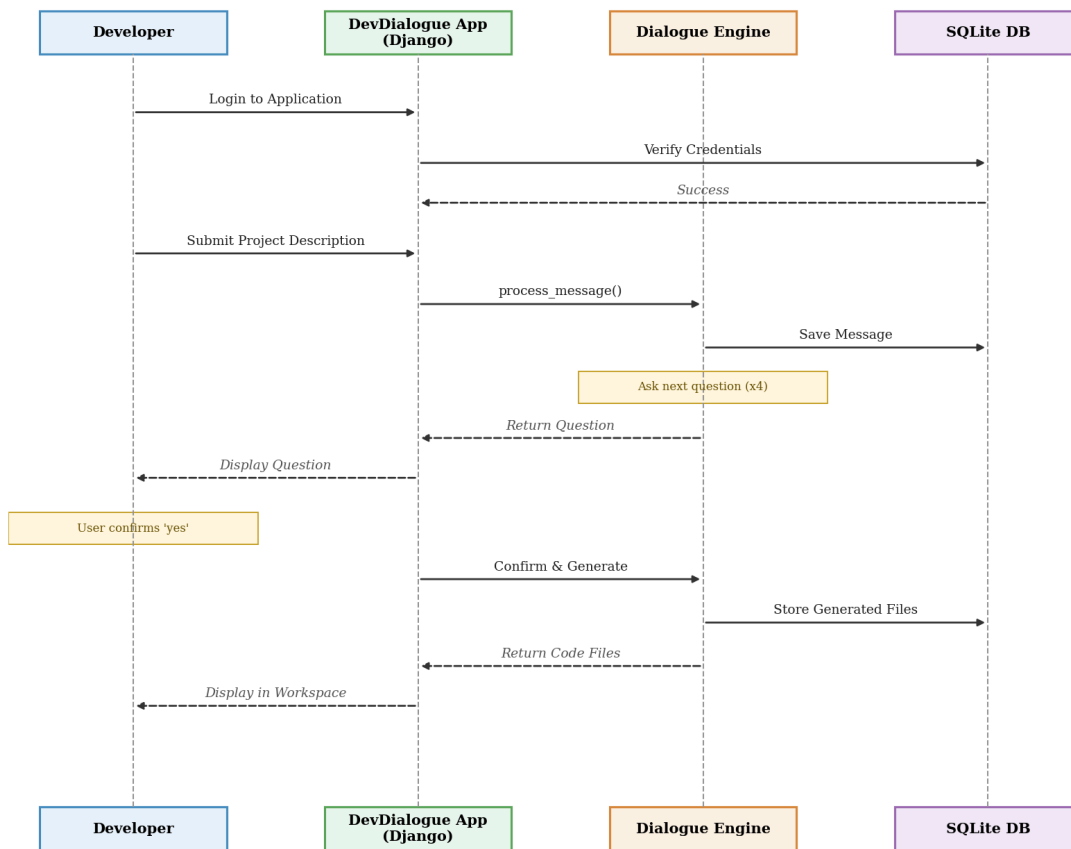


Figure 5.5 Sequence diagram — Mode 1 dialogue and generation

The developer logs in and the credentials are verified before any work begins.

The developer submits a project description, which the view passes to the dialogue engine.

The engine saves each message and returns the next question until all four are answered.

On confirmation, the engine generates and stores the files, which are returned to the workspace.

5.3.5 Activity Diagram

Figure 5.6 shows the flow from entering a description through the question loop, the confirmation checkpoint, and the mode-specific generation paths, including the per-module approval loop of Mode 2.

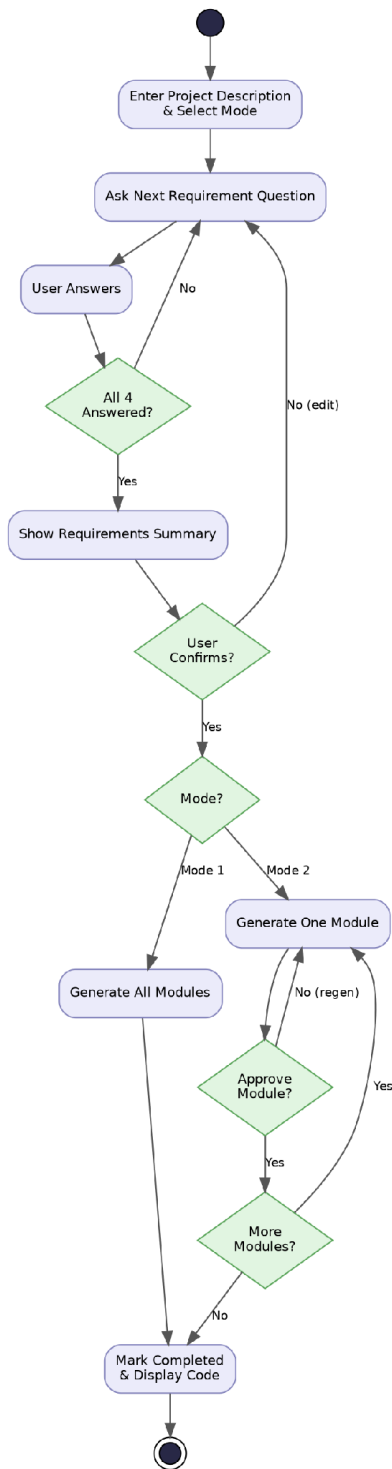


Figure 5.6 Activity diagram — dialogue and generation workflow

The flow begins when the developer enters a project description and selects a mode.

The engine asks each requirement question in turn until all four are answered.

The developer reviews the summary and either confirms or requests further changes.

On confirmation, Mode 1 generates all modules at once, while Mode 2 loops through per-module approval.

The session is marked completed and the generated code is displayed.

5.3.6 State-Transition Diagram

Figure 5.7 shows the session lifecycle: active collection, awaiting confirmation, generation (iterating per module in Mode 2), and completed.

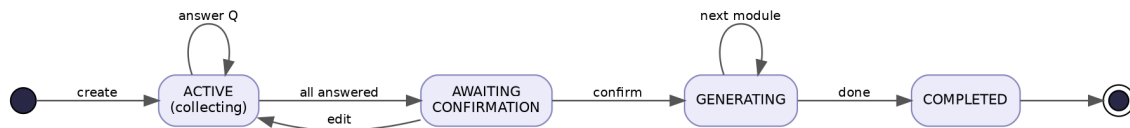


Figure 5.7 Session state-transition diagram

A session begins in the ACTIVE state while requirements are being collected.

Once all questions are answered it moves to AWAITING CONFIRMATION.

On confirmation it enters GENERATING, iterating per module in Mode 2.

When generation finishes the session becomes COMPLETED and is closed to further messages.

5.7 Design Summary

The design delivers each functional requirement through a small number of cooperating components arranged in three clean layers, with all core behaviour concentrated in the dialogue engine. The static models define four domain entities and their relationships; the behavioural models define a clear question-to-generation workflow and a simple, well-bounded session lifecycle; and the security design layers verification, OTP integrity, a strong password policy, and access control across the whole application. Critically, the isolation of the generation stage behind a single interface means the present template-based engine and a future large-language-model backend are interchangeable, so the elicitation and workflow design documented here remains valid as the generation capability evolves.

5.4 Data Design

The information domain is realised as four related tables managed by the ORM, alongside Django's User table. A Profile extends each User one-to-one; a Session belongs to a User and aggregates its Messages and GeneratedCode records. Referential integrity uses cascading deletes. The entity-relationship diagram is shown in Figure 5.8.

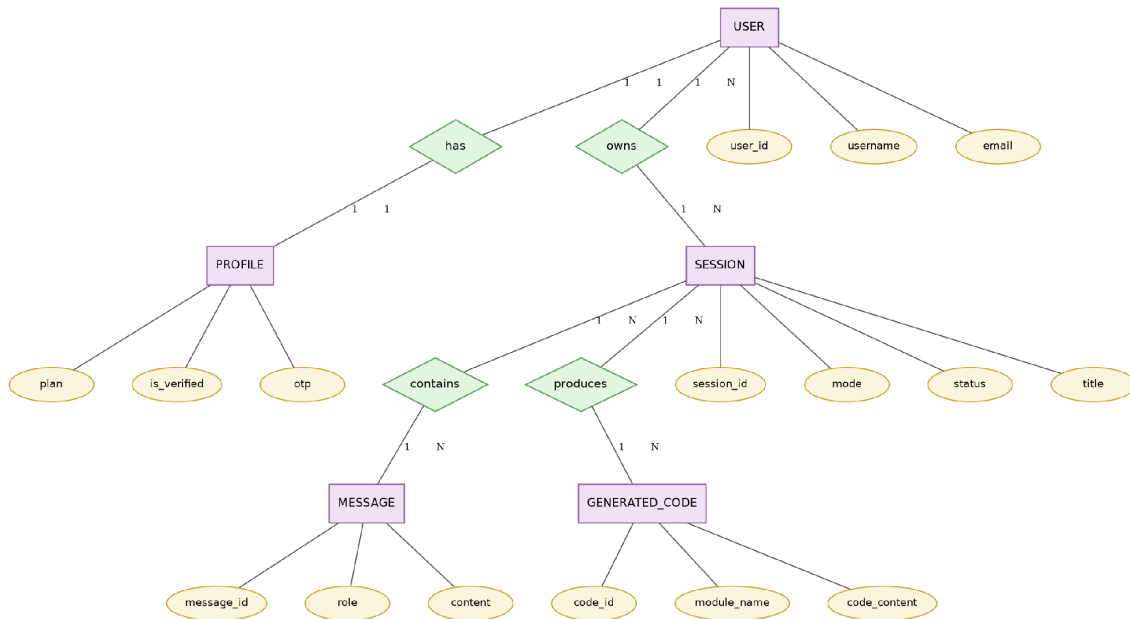


Figure 5.8 Entity-relationship diagram

A User has exactly one Profile (a one-to-one relationship).

A User owns many Sessions (a one-to-many relationship).

Each Session contains many Messages, stored in order.

Each Session produces many GeneratedCode records, one per generated module.

Cascading deletes preserve integrity: removing a session removes its messages and generated files.

5.4.1 Data Dictionary

Profile

Attribute	Type	Description
user	OneToOne(User)	Owning user (auto-created via post_save signal)
plan	Char (basic/pro/premium)	Subscription tier; defaults to basic
is_verified	Boolean	Whether the email has been verified
otp	Char(6), nullable	Current one-time password
otp_created_at	DateTime, nullable	Timestamp enforcing the two-minute OTP expiry
profile_picture	Image, nullable	Optional avatar upload

Session

Attribute	Type	Description
user	FK(User)	Owner of the session

mode	Char (mode1/mode2)	Selected generation workflow
status	Char (active/completed)	Lifecycle state
title	Char(255)	Human-readable title
has_custom_title	Boolean	Whether the title was explicitly set
started_at	DateTime	Creation timestamp, used for history grouping

Message

Attribute	Type	Description
session	FK(Session)	Parent session
role	Char (user/assistant)	Author of the message
content	Text	Message body
timestamp	DateTime	Creation time; messages ordered by this field

GeneratedCode

Attribute	Type	Description
session	FK(Session)	Parent session
module_name	Char(100)	Logical file name (e.g. models.py)
code_content	Text	The generated file contents
generated_at	DateTime	Generation timestamp

5.5 Security Design

Four mechanisms work together. Account verification: a new account is inactive until the user proves email ownership with a six-digit OTP. OTP integrity: each OTP is a random six-digit code stored with a timestamp, accepted only if it matches and is within a two-minute window, with a resend path. Password strength: a custom validator requires a minimum length, a capitalised first letter, and lower-case, digit, and special characters, governing both registration and reset. Access control and request integrity: a verification-required decorator guards every authenticated view, while Django CSRF protection secures every state-changing request.

5.6 User Interface Design

5.6.1 Screen Images and Flow

The interface uses a familiar conversational metaphor. New users complete a sign-up form, receive an OTP by email, and confirm it on a verification screen with a live two-minute countdown and resend option. After verification the dashboard lets them write a project

description, choose Mode 1 or Mode 2, and launch a session. The chat workspace shows past sessions in a left sidebar, the conversation in the centre, and a collapsible, tabbed code panel on the right. The interface supports dark and light themes, remembered between visits.

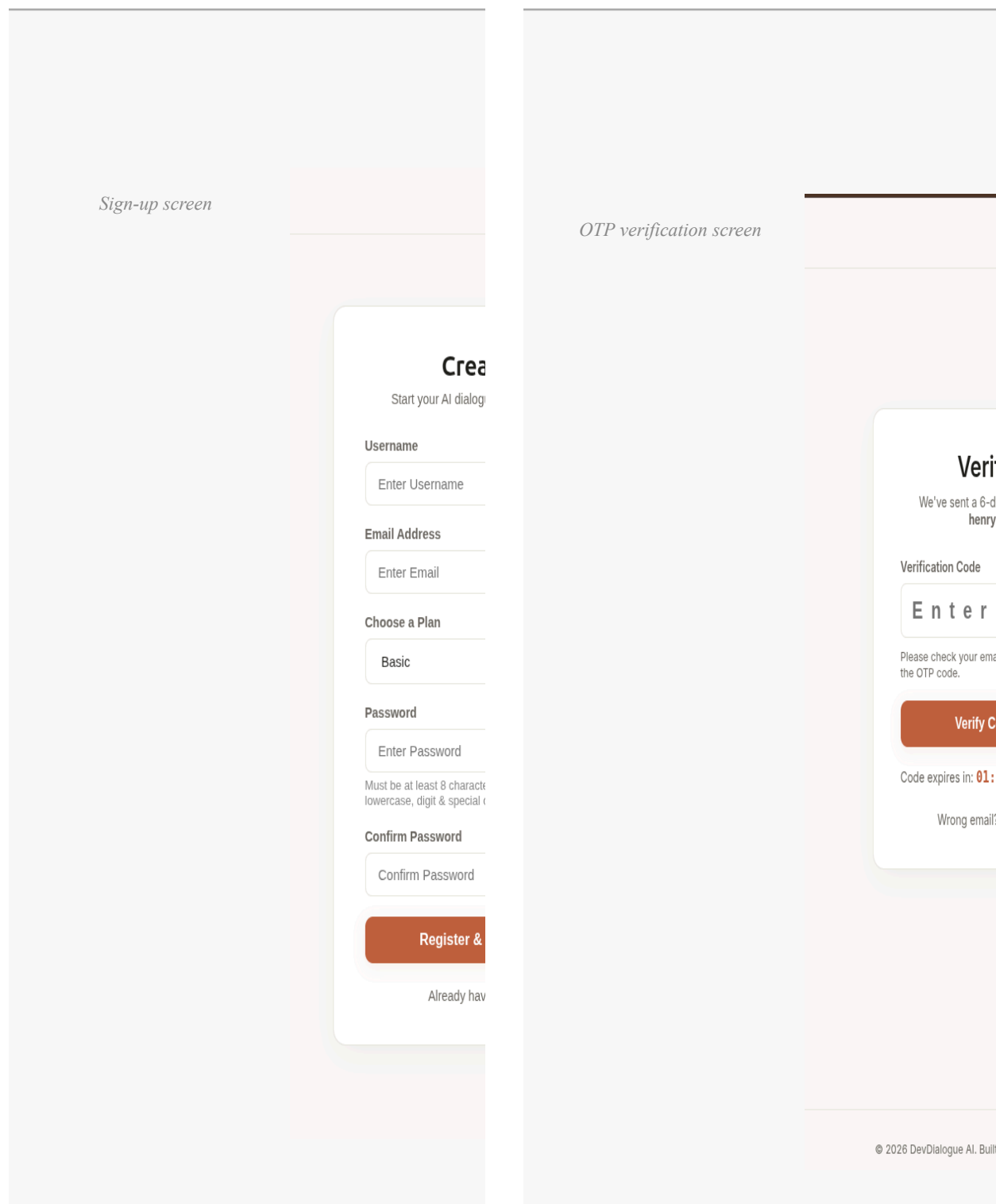


Figure 5.9 Registration and OTP verification screens

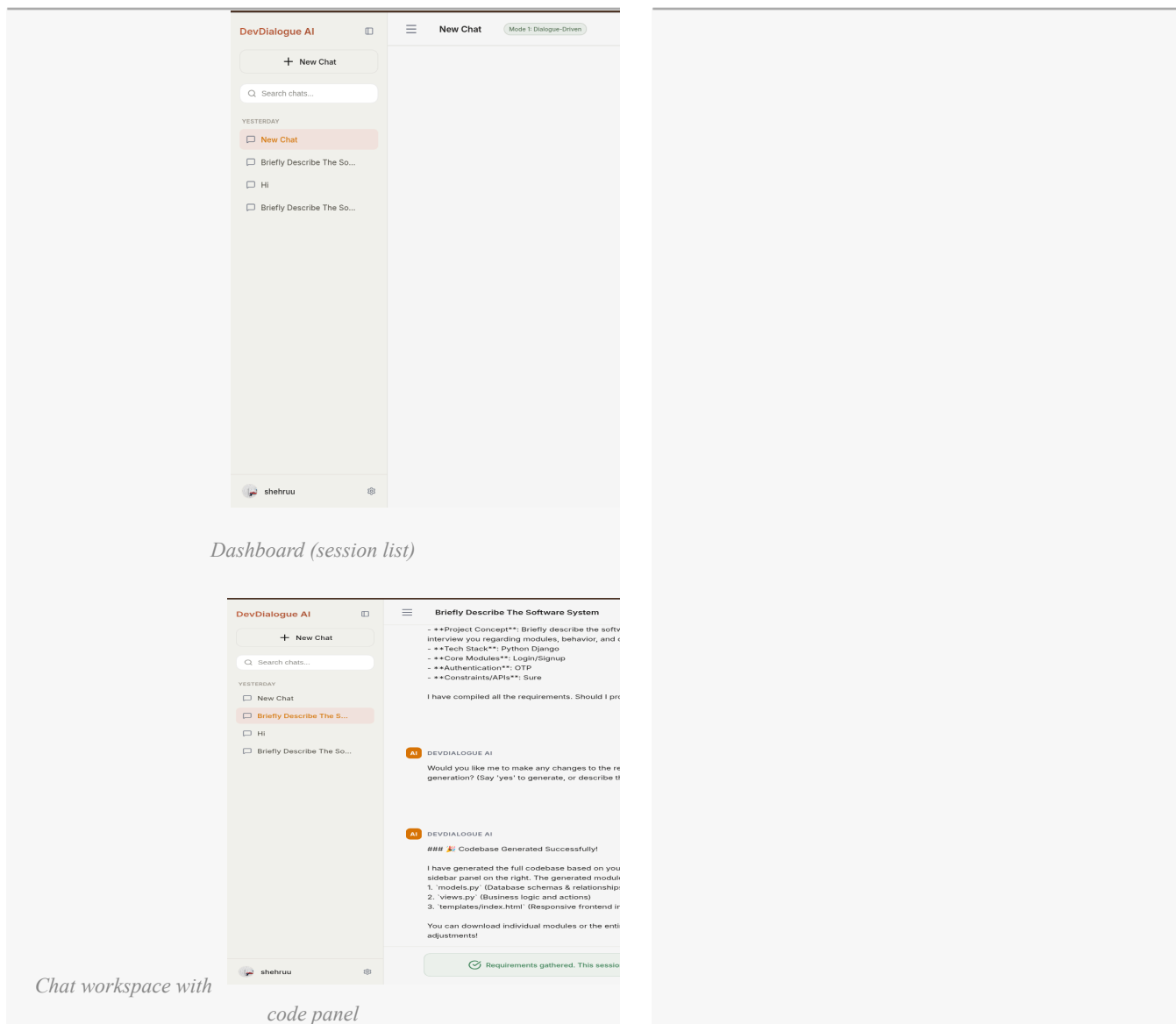


Figure 5.10 Dashboard and chat workspace

5.6.2 Screen Objects and Actions

Authentication screens: inputs for sign-up, login, OTP entry, and reset, with inline validation, an OTP countdown, and a resend action.

Dashboard: a mode selector, a project-description field, a launch action, and a list of previous sessions with resume/view actions.

Chat workspace: a message input with send action; message bubbles; a sidebar with rename, delete, and search; and a tabbed code panel with view, copy, and download.

Settings: profile update (username, avatar) and change-password actions.

Chapter 6

Implementation and Testing

This chapter describes how DevDialogue AI was implemented and how it was tested. It opens with a discussion of the implementation approach and the tools used, then summarises the module structure, and finally presents the testing strategy and the detailed test cases and their results.

6.1 Implementation Discussion

The central implementation challenge was to build a system in which a conversational front end, a stateful session model, and a generation stage cooperate cleanly while remaining loosely coupled. The guiding principle throughout was to keep the views thin and to concentrate the product's distinctive behaviour, the requirement dialogue and the two generation modes, inside a single engine component. This separation paid off repeatedly during development: because the engine could be exercised independently of the web layer, its behaviour could be refined and tested without touching the interface, and the interface could be reshaped without risk to the generation logic.

Security was treated as a first-class concern from the first development cycle rather than as a finishing touch. Building the account subsystem first meant that every later feature was developed behind real authentication, so the verification-required behaviour and the OTP flows were exercised continuously. This ordering surfaced edge cases, such as the handling of expired codes and the redirection of unverified users, early enough to address them cleanly.

6.2 Tools and Technologies

The implementation uses a focused, open-source stack. The table below lists the principal tools and their role in the project.

Tool / Technology	Role in the project
Python 3	Primary implementation language
Django 6.0	Web framework providing the MVT structure, ORM, auth, and mail
SQLite	Development database, accessed through the ORM
HTML5, CSS3, JavaScript	The front-end templates, styling, and asynchronous behaviour
Django TestCase	The automated unit and integration test framework
Git and GitHub	Version control and collaboration
Visual Studio Code	The development environment

SMTP (TLS)	Delivery of OTP and password-reset emails
------------	---

Table 6.1 Tools and technologies

6.3 Module Structure

The system is implemented as a Django 6.0 project. The project package (config) holds the settings, URL configuration, and WSGI/ASGI entry points, while a single application (core) holds the domain logic. The core application is organised into clear modules: models define the four domain entities; views handle every HTTP and AJAX request; forms and validators enforce input rules; the dialogue engine encapsulates the requirement-elicitation and generation behaviour; and a decorator enforces verified access. Templates and static assets provide the interface.

Module	Responsibility
config/settings.py	Project configuration: apps, database, email/SMTP, static and media, auth redirects
core/models.py	Profile, Session, Message, and GeneratedCode entities plus the profile-creation signal
core/views.py	Home, auth, OTP, dashboard, session, chat, and code-retrieval endpoints
core/dialogue_engine.py	Question flow, requirements summary, and Mode 1 / Mode 2 generation
core/forms.py	Sign-up, OTP, forgot/reset-password, and profile-update forms
core/validators.py	Custom password-strength validator
core/decorators.py	verification_required access-control decorator
core/templates/	Base layout, landing pages, auth screens, dashboard, and chat workspace

Table 6.2 Core module structure

6.3.1 Authentication and Account Management

The authentication subsystem is built on Django's own authentication framework, extended with an email-verification layer. When a visitor submits the sign-up form, the account is created inactive and a Profile record is created automatically through a post-save signal. A six-digit one-time password is generated, stored with a timestamp, and emailed to the user through the configured SMTP service. The verification screen accepts the code only while it is within its two-minute window, and offers a resend action once it expires. Password strength is enforced by a custom validator that requires a minimum length together with an upper-case first letter and at least one lower-case letter, digit, and special character; the same validator governs both registration and password reset. A verification-required decorator wraps every authenticated view so that unauthenticated users are redirected to login and unverified users to the verification screen.

6.3.2 Sessions, Dialogue, and Generation

Each project workspace is a Session record that stores its mode, status, and title. Messages are stored in order and replayed to rebuild the conversation whenever a session is reopened. The dialogue engine reads the message history to decide how far the interview has progressed: while fewer than four answers have been given it returns the next question, and once all four are answered it assembles a consolidated summary and asks for confirmation. On confirmation it writes the generated files as GeneratedCode records, one per module, and marks the session completed. In Mode 2 it produces one module at a time, storing and presenting each before continuing, and regenerates a module with the user's feedback if changes are requested.

6.3.3 Front-End and Code Workspace

The interface is rendered with Django templates and progressively enhanced with vanilla JavaScript. Chat messages are sent with asynchronous fetch requests so the conversation updates without a full page reload, and the generated files are presented in a tabbed, collapsible panel that supports viewing, copying, and downloading each file. The design system provides both dark and light themes, and the session sidebar groups past work by recency and supports client-side search.

6.4 Testing Strategy

Testing followed a layered strategy. Unit tests verify individual components in isolation, the OTP lifecycle, the password validator, and the forms, using Django's test framework. Integration tests exercise the dialogue engine end-to-end, confirming that a full sequence of answers advances through the questions, produces a summary, and generates the expected files. System and acceptance testing were performed manually against a running instance with a configured SMTP account, covering the complete user journeys of registration, verification, login, session creation, both generation modes, and password reset. The following sections give representative test cases and their results.

6.5 Unit Test Cases

These test cases validate individual components. They correspond to the automated unit tests in the project's test suite.

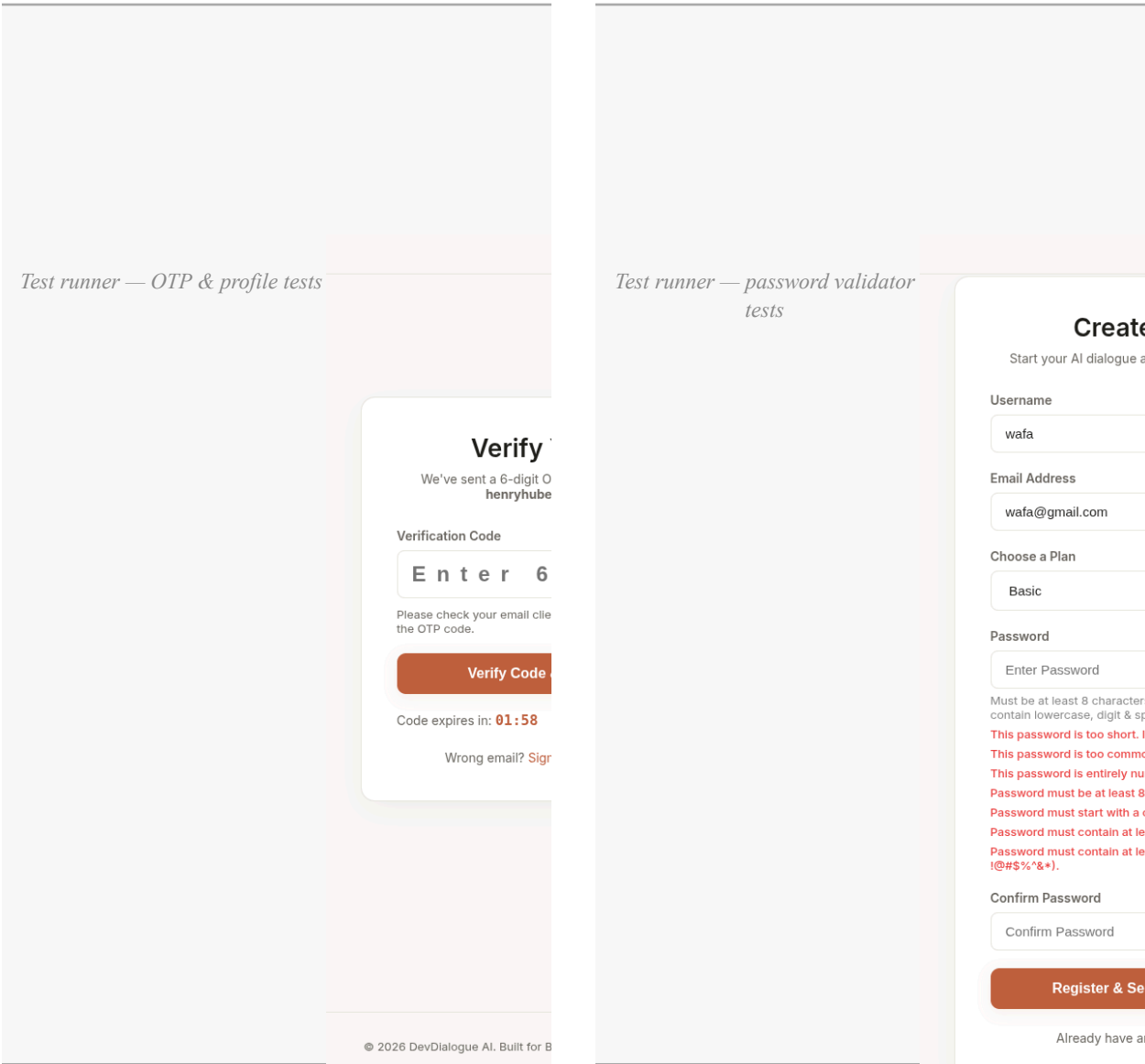


Figure 6.1 Unit test execution output

ID	Test Case	Steps / Input	Expected Result	Status
TC-01	Automatic profile creation	Create a new User object	A Profile is auto-created with plan	Pass

			= basic and is_verified = false	
TC-02	OTP generation	Call generate_otp() on a profile	A random 6-digit code is stored with a creation timestamp	Pass
TC-03	Valid OTP accepted	Submit the correct OTP within 2 minutes	is_otp_valid() returns true	Pass
TC-04	Wrong OTP rejected	Submit an incorrect OTP	is_otp_valid() returns false	Pass
TC-05	Expired OTP rejected	Submit a correct OTP after 3 minutes	is_otp_valid() returns false	Pass

Table 6.2 Unit test cases — OTP and profile

6.5.1 Password Policy Test Cases

ID	Test Case	Steps / Input	Expected Result	Status
TC-06	Strong password accepted	Validate "Password123!"	No validation error is raised	Pass
TC-07	Lower-case first letter rejected	Validate "password123!"	Error: must start with a capital letter	Pass
TC-08	Too-short password rejected	Validate "Pass!"	Error: must be at least 8 characters	Pass
TC-09	Missing digit rejected	Validate "Password!"	Error: must contain a digit	Pass
TC-10	Missing special char rejected	Validate "Password123"	Error: must contain a special character	Pass
TC-11	Sign-up form rejects weak password	Submit sign-up with "invalidpassword"	Form is invalid; password error shown	Pass
TC-12	Reset form rejects weak password	Submit reset with "weak"	Form is invalid; new-password error shown	Pass

Table 6.3 Unit test cases — password policy and forms

6.6 Integration Test Cases

These test cases validate the dialogue engine and the generation flow end-to-end, corresponding to the automated dialogue-flow test.

ID	Test Case	Steps / Input	Expected Result	Status
TC-13	Question sequence	Submit answers one by one	Engine asks Q1→Q2→Q3→Q4 in order	Pass

TC-14	Requirements summary	Answer all four questions	A consolidated summary is produced and confirmation requested	Pass
TC-15	Mode 1 generation	Confirm with "yes"	Three files are generated (models, views, index) and session marked completed	Pass
TC-16	Generated file set	Inspect generated_codes	Modules include models.py, views.py, templates/index.html	Pass
TC-17	Mode 2 per-module	Approve each module in turn	Modules are generated one at a time with approval between each	Pass

Table 6.4 Integration test cases — dialogue and generation

6.7 System / Acceptance Test Cases

These test cases were executed manually against a running instance and cover the full user journeys.

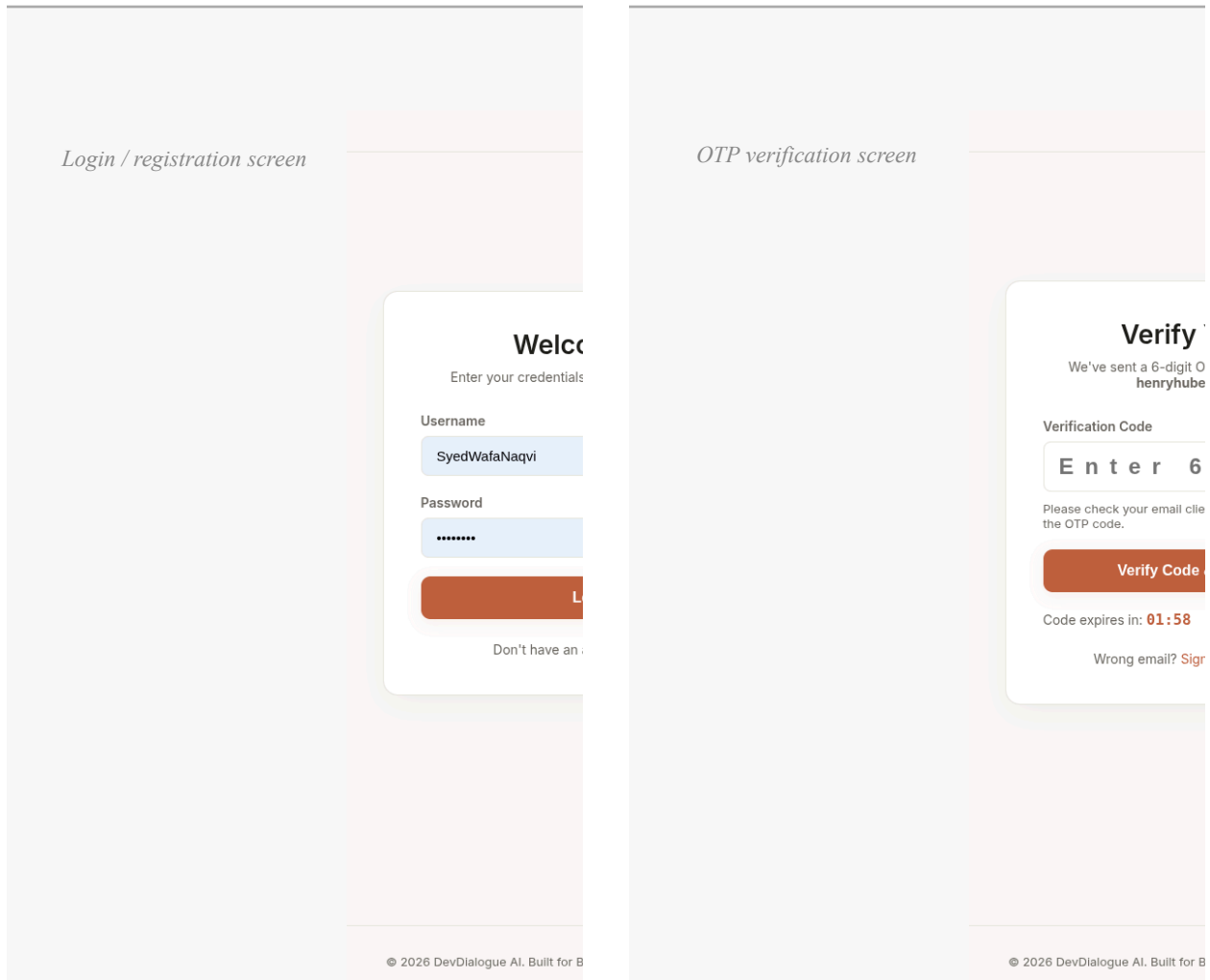


Figure 6.2 Authentication screens under test

ID	Test Case	Steps / Input	Expected Result	Status
TC-18	Registration & email	Register with valid details	Account created inactive; OTP email received	Pass
TC-19	Account verification	Enter the correct OTP	Account activated and user logged in to the dashboard	Pass

TC-20	OTP resend	Wait for expiry, click resend	A new OTP is emailed and the countdown restarts	Pass
TC-21	Login (verified)	Log in with valid credentials	User is authenticated and redirected appropriately	Pass
TC-22	Login (unverified)	Log in before verifying	User is redirected to OTP verification	Pass
TC-23	Password reset	Complete the forgot/reset flow with OTP	Password is updated; user can log in with the new password	Pass
TC-24	Session create	Launch a session from the dashboard	A new session opens in the chat workspace	Pass
TC-25	Code workspace	Generate files and open the panel	Files appear as tabs; view, copy, and download work	Pass
TC-26	Rename & delete	Rename then delete a session	Title updates; session is removed from history	Pass
TC-27	Access control	Open a workspace URL while logged out	User is redirected to login	Pass

Table 6.5 System and acceptance test cases

6.8 Test Summary

Across the three levels, 27 representative test cases were executed: 12 unit, 5 integration, and 10 system/acceptance. All passed. The automated unit and integration tests run through Django's test runner, so they can be re-executed on every change to guard against regressions, while system tests confirm that the complete user journeys behave as specified. Together they validate the functional requirements FR-01 to FR-17 and the security-related non-functional requirement NFR-02.

Test Level	Cases	Passed	Failed	Pass Rate
Unit (OTP, profile, password, forms)	12	12	0	100%
Integration (dialogue, generation)	5	5	0	100%
System / Acceptance (user journeys)	10	10	0	100%
Total	27	27	0	100%

Table 6.6 Test execution summary

Chapter 7

Conclusion and Future Work

This chapter summarises what the project set out to do, what was achieved, and where the work can be taken next. It reflects on the extent to which the aims of Chapter 1 were met and identifies the concrete extensions that would move the system from its current, requirement-focused implementation to a fully model-driven product.

7.1 Conclusion

DevDialogue AI was undertaken to test a simple but under-explored idea: that the quality of AI-assisted code generation can be improved not only by using a more capable model, but by placing a structured requirement-elicitation conversation ahead of generation. The project delivered a working web application that embodies this idea. A deterministic dialogue engine interviews the developer across the four dimensions most often left unstated, technology stack, core modules, authentication, and constraints, consolidates the answers into a reviewable summary, and generates project files only after the developer confirms that summary. Two complementary workflows, full generation and incremental per-module generation, are unified under a single session model.

Around this core, the project delivered the supporting subsystems that make it a coherent product rather than a proof of concept: secure registration with email OTP verification, password reset, a custom password policy, persistent chat-style sessions with searchable history, and a code workspace that presents each generated file for viewing, copying, and downloading. The system was validated through twenty-seven test cases spanning unit, integration, and system levels, all of which passed.

Measured against the objectives set out in Chapter 1, the project met its aims. The requirement-elicitation engine and the two generation modes were designed and implemented; the secure, production-shaped web application was built; the generated output is presented in an accessible workspace; and the generation stage was deliberately isolated behind a single interface so that it can evolve independently of the rest of the system. The main capability held back for future work, live large-language-model generation, was consciously scoped out and replaced by a deterministic engine that occupies exactly the interface a model would later fill.

7.2 Limitations

The current version has several limitations that follow naturally from its scope. Generation is deterministic and template-based rather than model-driven, so the variety and depth of the

generated code are bounded by the templates the engine contains. The requirement interview is a fixed four-question sequence, which covers the most common gaps but does not yet adapt its questioning to the specifics of an unusual project. The platform targets Python and general web-development projects and does not attempt to cover every language or framework. It requires an internet connection for email delivery, uses SQLite in development rather than a production database, and does not include advanced features such as real-time multi-user collaboration or an IDE plugin.

None of these limitations undermines the project's central contribution, which is the demonstration of the requirement-aware interaction pattern. Rather, they mark the boundary between what was necessary to demonstrate that pattern and what would be needed to turn it into a production system.

7.3 Future Work

The most immediate and impactful extension is to replace the deterministic generation method with a live large-language-model backend. Because the engine already isolates generation behind a single interface, this substitution can be made without disturbing the dialogue, session, or workspace layers; the elicited requirements summary would simply be passed to the model as structured context. An optional orchestration layer, such as an N8N workflow, could manage multi-step prompting and post-processing around that model.

Beyond the generation backend, several further extensions would strengthen the system. The requirement interview could be made adaptive, learning when enough context has been gathered and tailoring its questions to the project rather than following a fixed sequence. The persistence layer could be moved to a production database such as PostgreSQL to support concurrent use at scale. The platform could be extended with an IDE plugin so that the elicitation step is available where developers already work, and its language coverage could be broadened beyond its current focus. Finally, a feedback loop that records which generated files developers accept or revise would provide data to improve both the question set and the generation quality over time.

7.4 Closing Remarks

DevDialogue AI shows that a requirement-aware layer is a valuable and separable contribution to AI-assisted development. By making requirements explicit and reviewable before code is written, and by structuring the system so that the generation backend can evolve independently, the project offers both a working demonstration today and a clear path to a more powerful system tomorrow. The result is a foundation on which stronger generation can be layered without rethinking the interaction that makes it useful.

REFERENCES

The following references were consulted during the design, implementation, and documentation of DevDialogue AI.

- [1] IEEE, IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. New York: IEEE, 1998.
- [2] Django Software Foundation, “Django Documentation (Version 6.0),” 2025. [Online]. Available: <https://docs.djangoproject.com>
- [3] I. Sommerville, Software Engineering, 10th ed. Harlow, England: Pearson Education, 2016.
- [4] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 8th ed. New York: McGraw-Hill Education, 2015.
- [5] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.
- [6] DeepSeek-AI, “DeepSeek Coder: When the Large Language Model Meets Programming,” 2024. [Online]. Available: <https://deepseek.com>
- [7] GitHub, “GitHub Copilot Documentation,” 2025. [Online]. Available: <https://docs.github.com/copilot>
- [8] OpenAI, “ChatGPT,” 2025. [Online]. Available: <https://openai.com/chatgpt>
- [9] Google, “Gemini,” 2025. [Online]. Available: <https://gemini.google.com>
- [10] Mozilla Developer Network, “Using the Fetch API,” 2025. [Online]. Available: <https://developer.mozilla.org>
- [11] Python Software Foundation, “Python 3 Documentation,” 2025. [Online]. Available: <https://docs.python.org/3>
- [12] United Nations, “The 17 Sustainable Development Goals,” 2015. [Online]. Available: <https://sdgs.un.org/goals>